day01_SSH服务

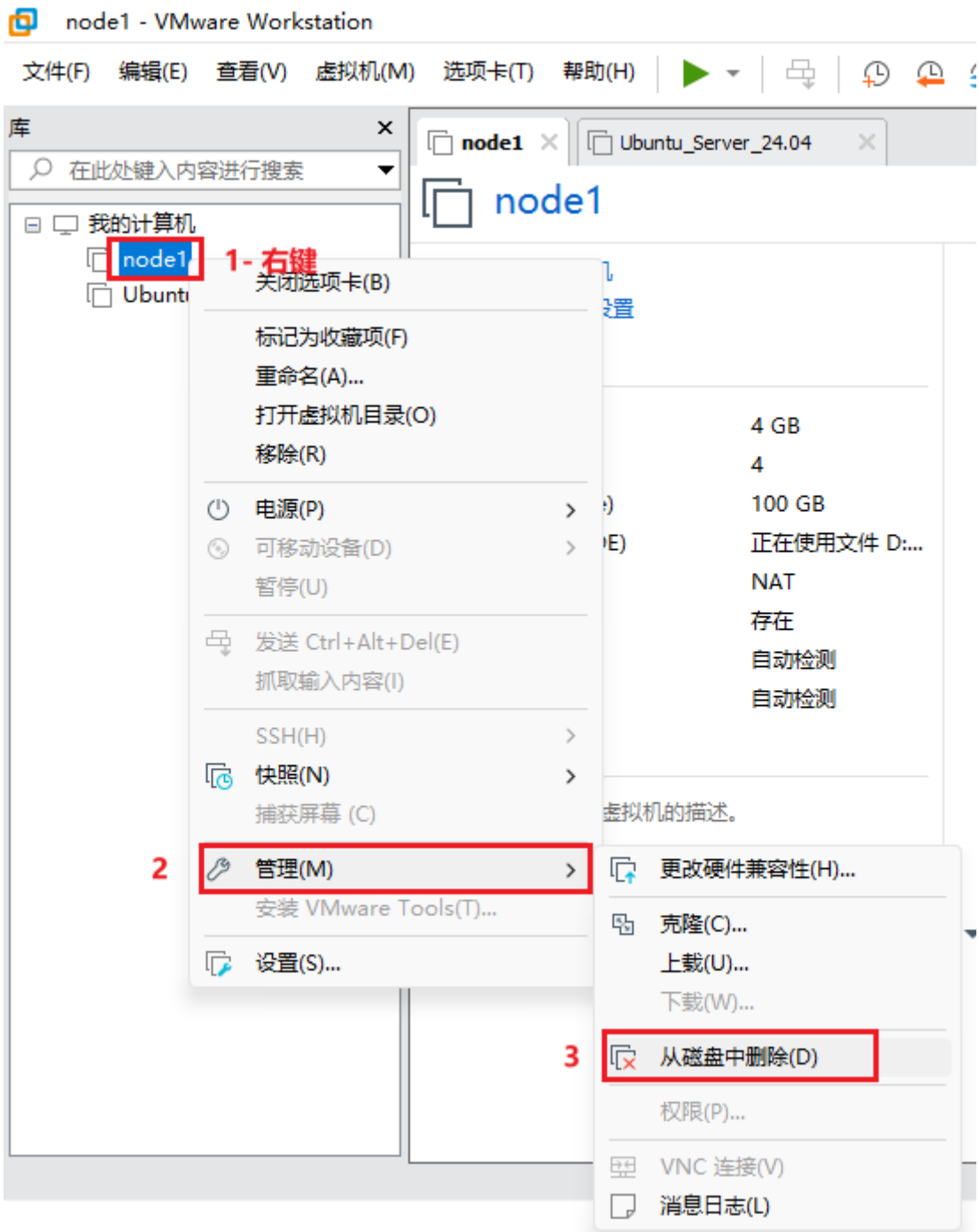
今日内容：

- 1-【VMware基本使用】如何克隆虚拟机：
  - 1.1 如何挂载一个新的虚拟机
  - 1.2 如何进行克隆虚拟机
- 2-【了解】SSH基本概念
- 3-【理解，能够讲的出来】SSH工作原理
- 4-【实操】SSH配置实战
  - 4.1 安装SSH相关服务
  - 4.2 基于SSH配置免密的远程登录方案
  - 4.3 基于SSH完成远程命令执行
  - 4.4 基于SSH进行免密数据传输
- 5-【实操】SSH综合案例
- 6-【实操】Ubuntu完成SSH服务配置

0. 如何挂载虚拟机

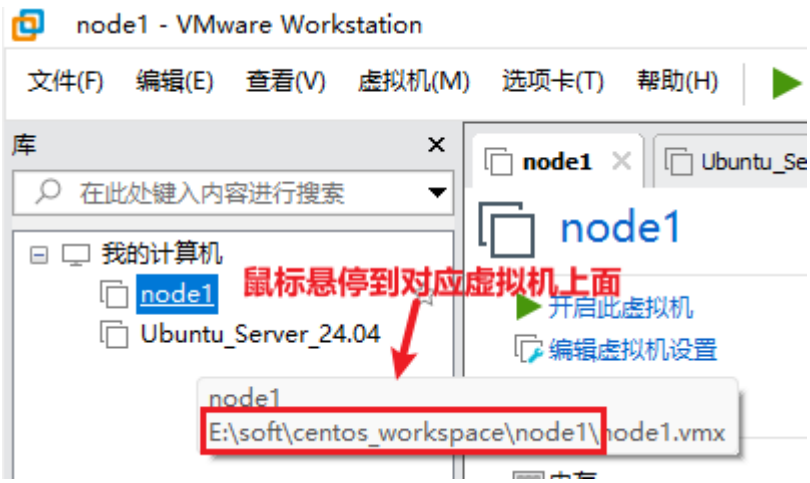
0.1 如何删除原有的虚拟机

方式一： 直接删除：



方式二：先移除，然后择机删除

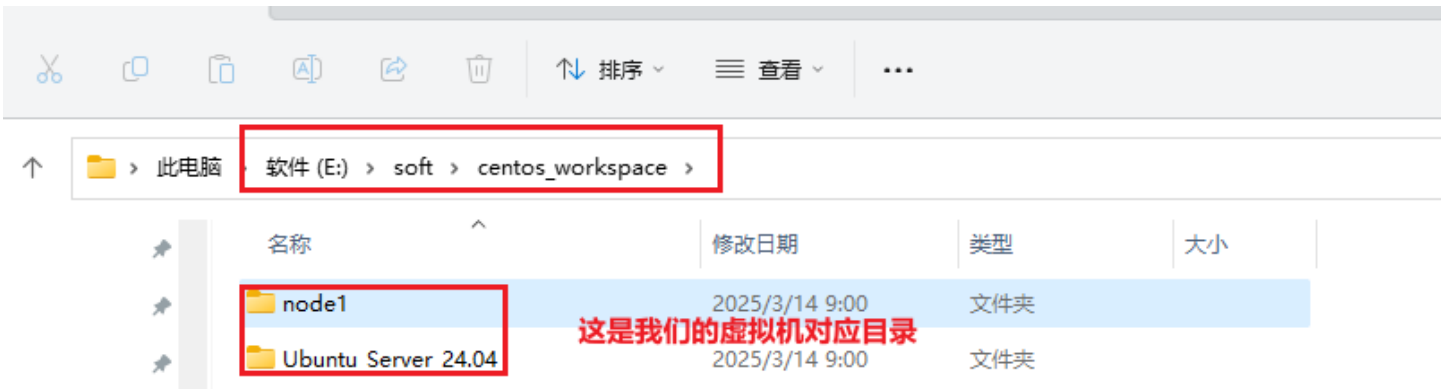
在移除前， 可以先查看一下，该虚拟机的所在位置：



说明： 如果位置是在一个磁盘根路径， 并没有单独放置到某一个文件中，建议直接选择方式一， 直接删除即可

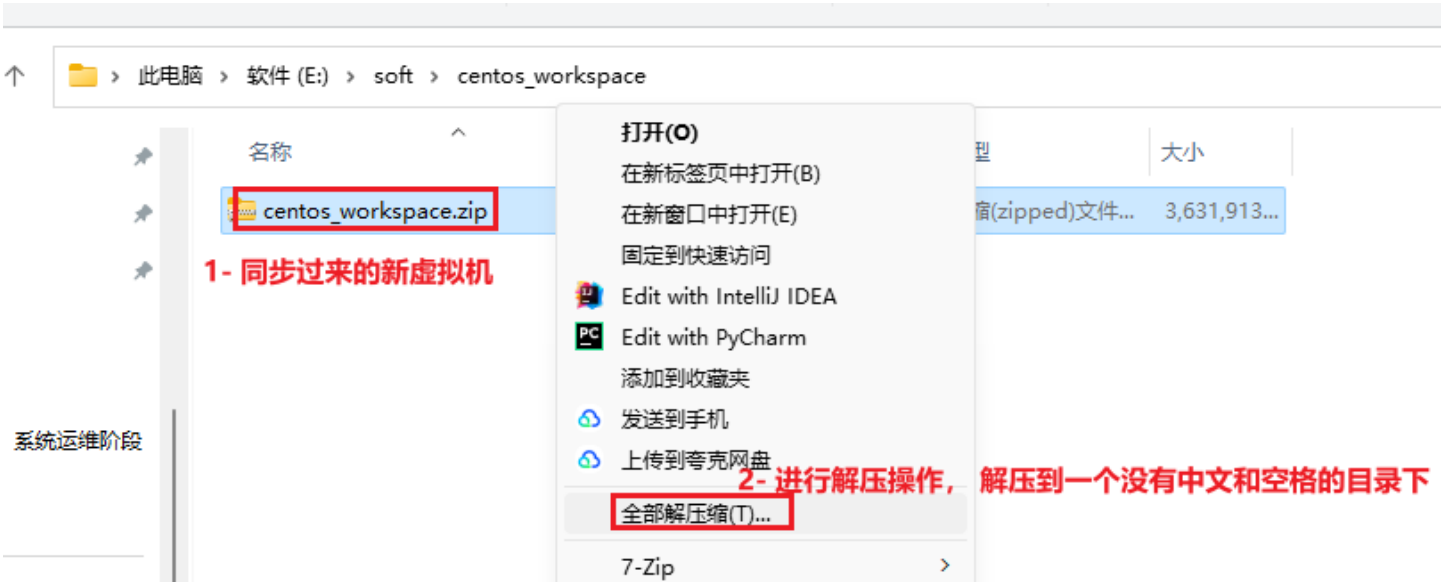


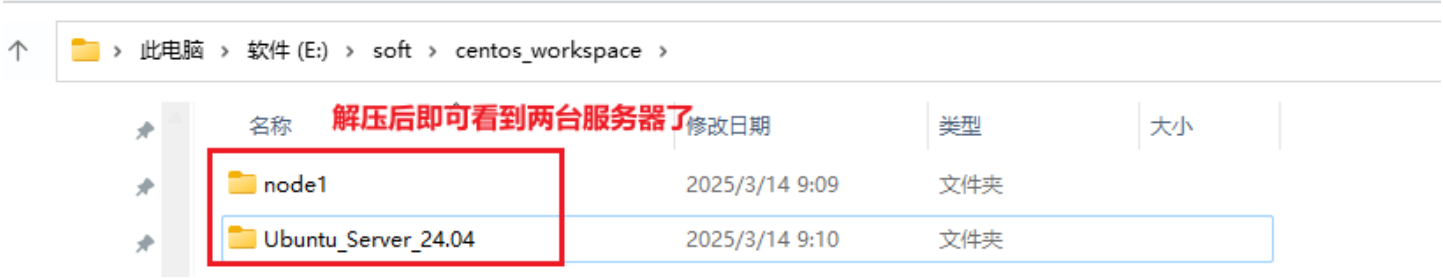
移除后，可以进入到该虚拟机的对应目录下



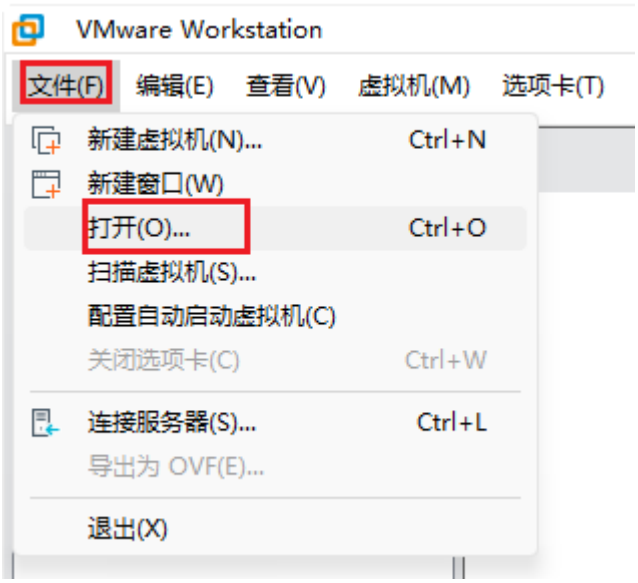
此时可以选择， 直接将这些目录删除， 既删除了虚拟机， 如果想要保留， 可以将这些移动到需要备份的地方即可

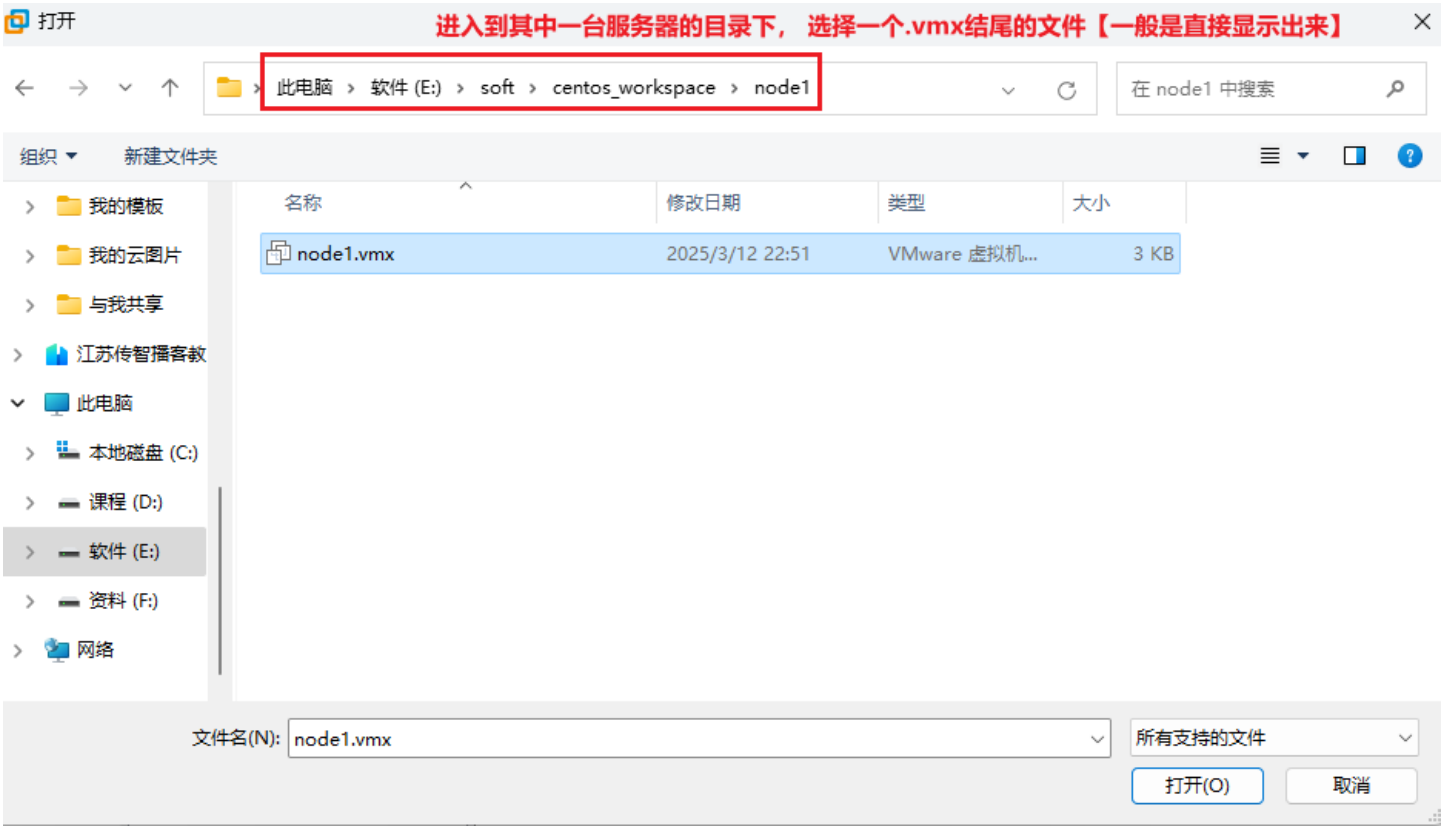
0.2 挂载新的提供虚拟机





接下来， 打开VMware：





同理， 另一个服务器也是类似的



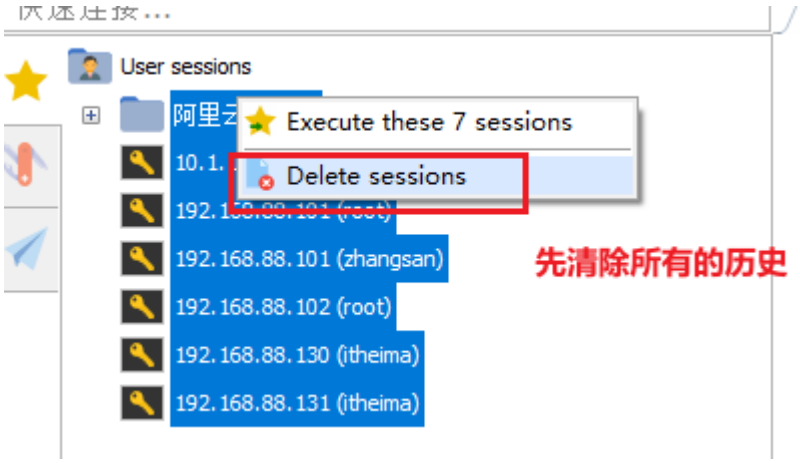
接下来， 直接开机了：

此时， 会弹出一个框框， 框框中有三个选项： 我以移动该虚拟机、我以复制该虚拟机、取消

请注意：此时务必选择，我以移动该虚拟机【思考：这是从你自己的虚拟机复制出来的吗？不是，所以选择我以移动】

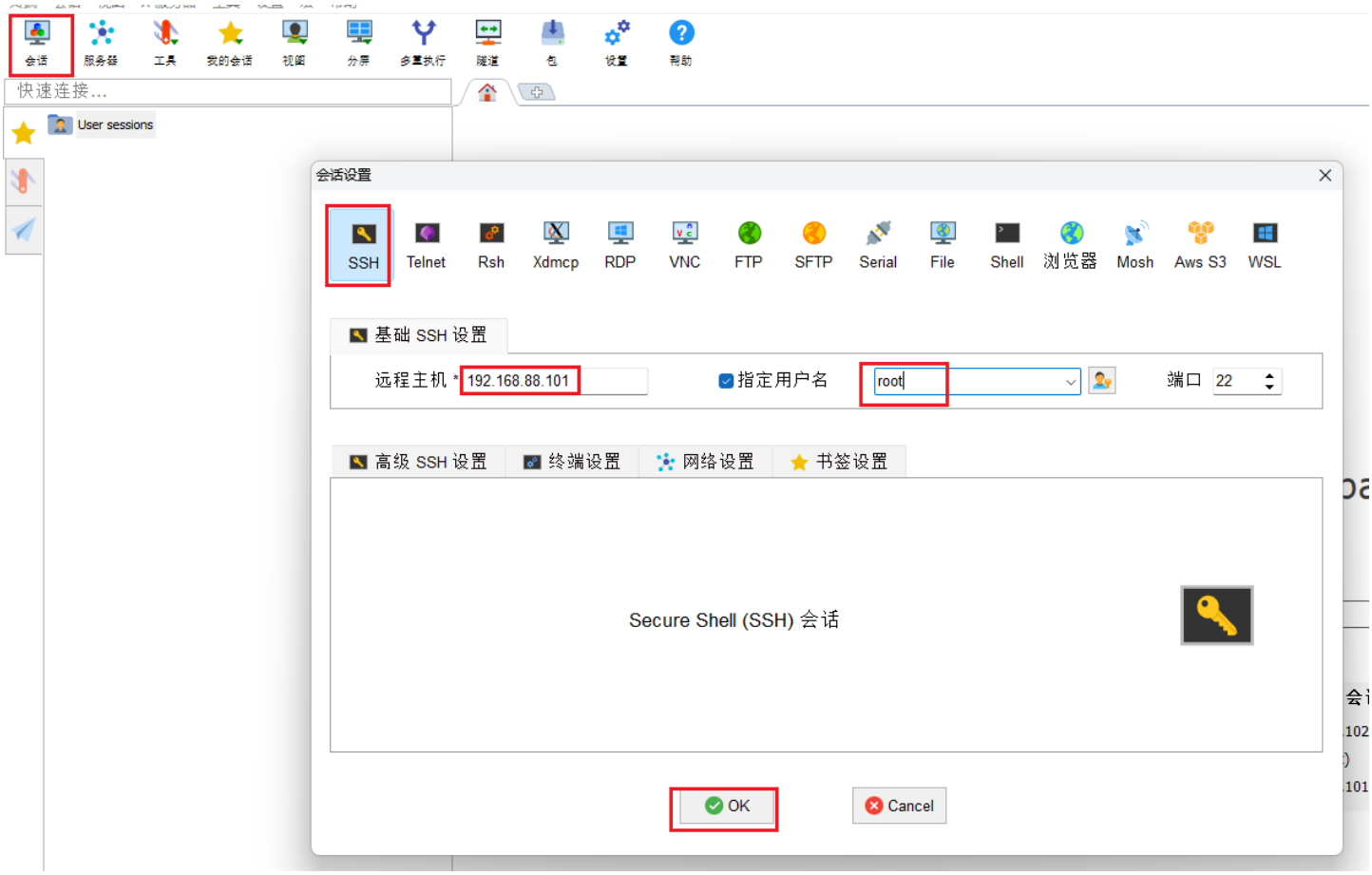
0.3 尝试使用MX工具连接服务器

由于，我们提供的虚拟机，与大家的所处的网段都是一致的， 所以开机后， 即可使用MX工具连接，但是如果在后续， 大家拿到是其他人的虚拟机， 有可能网络环境与你当前的网络环境不一致， 此时需要先修改虚拟机的网络环境【静态IP地址、网关、子网掩码、DNS地址 】

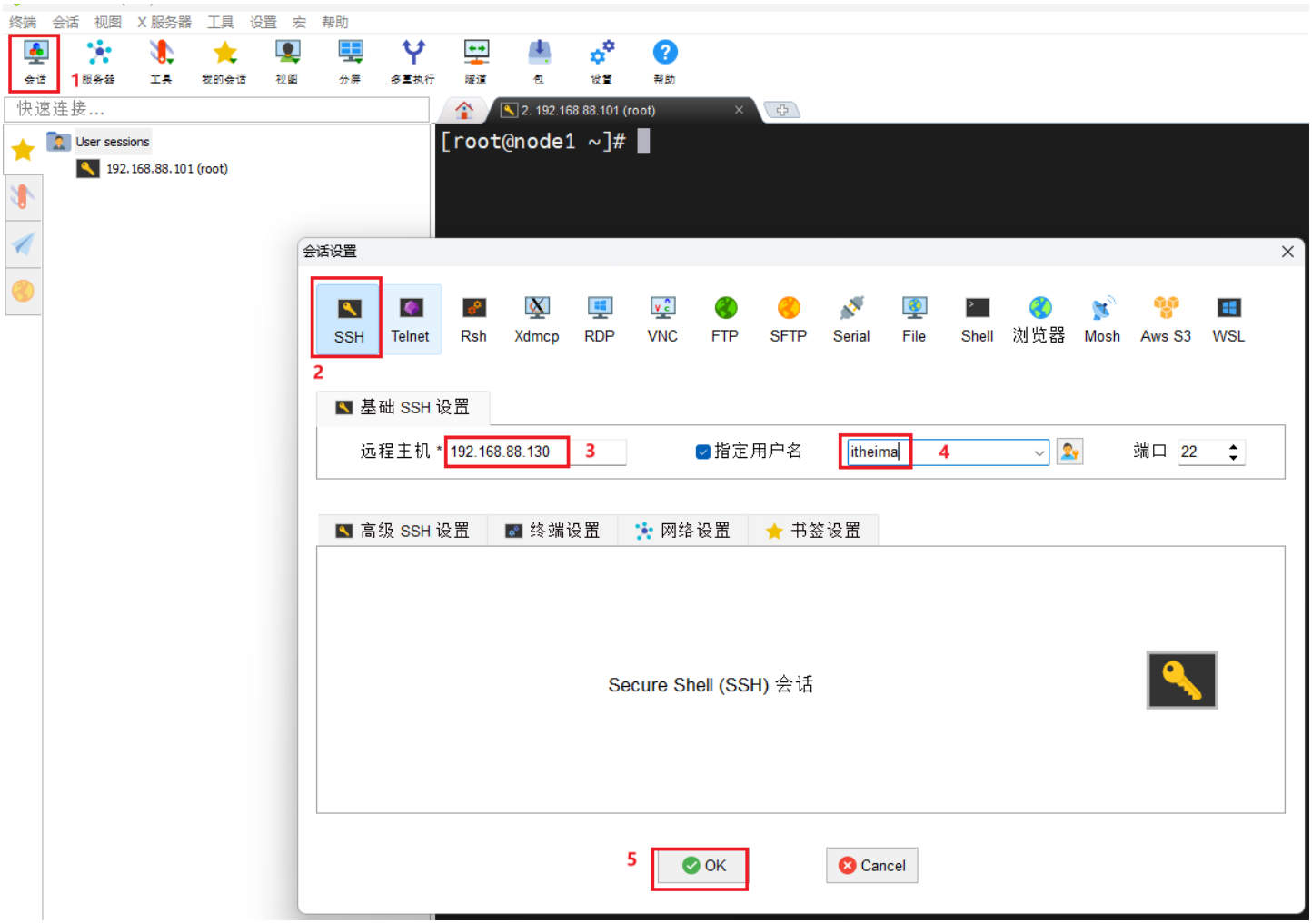


重新连接：

- 连接centos stream 9

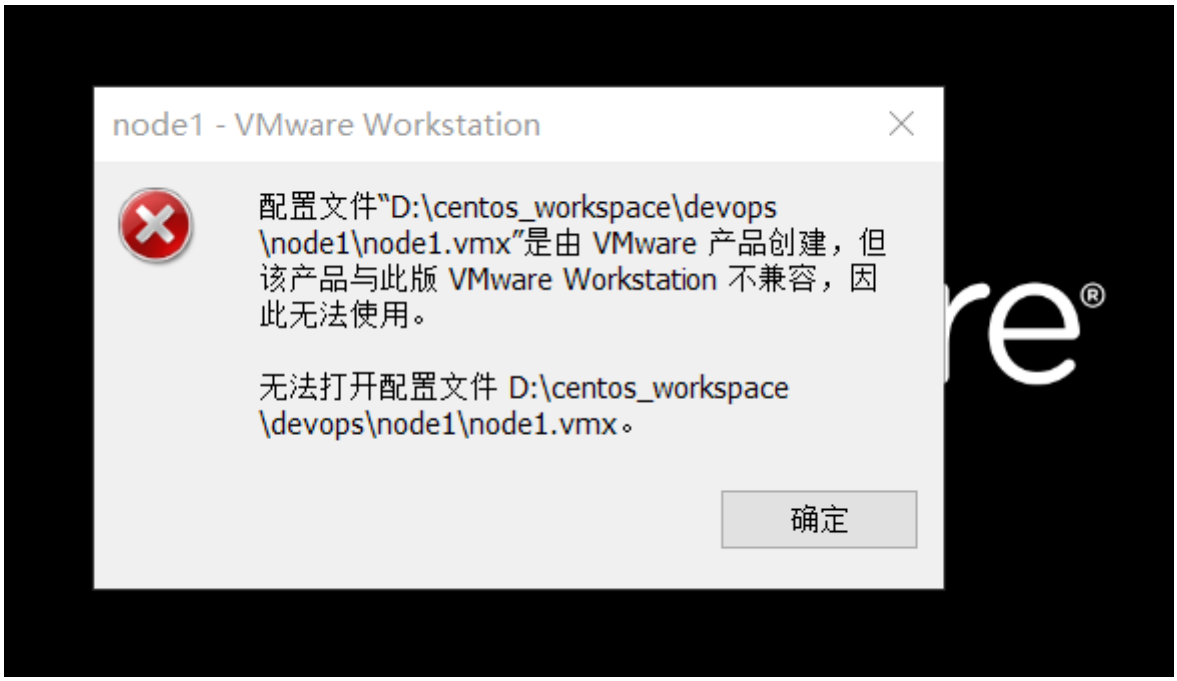


- Ubuntu Server 24.04



0.4 常见问题

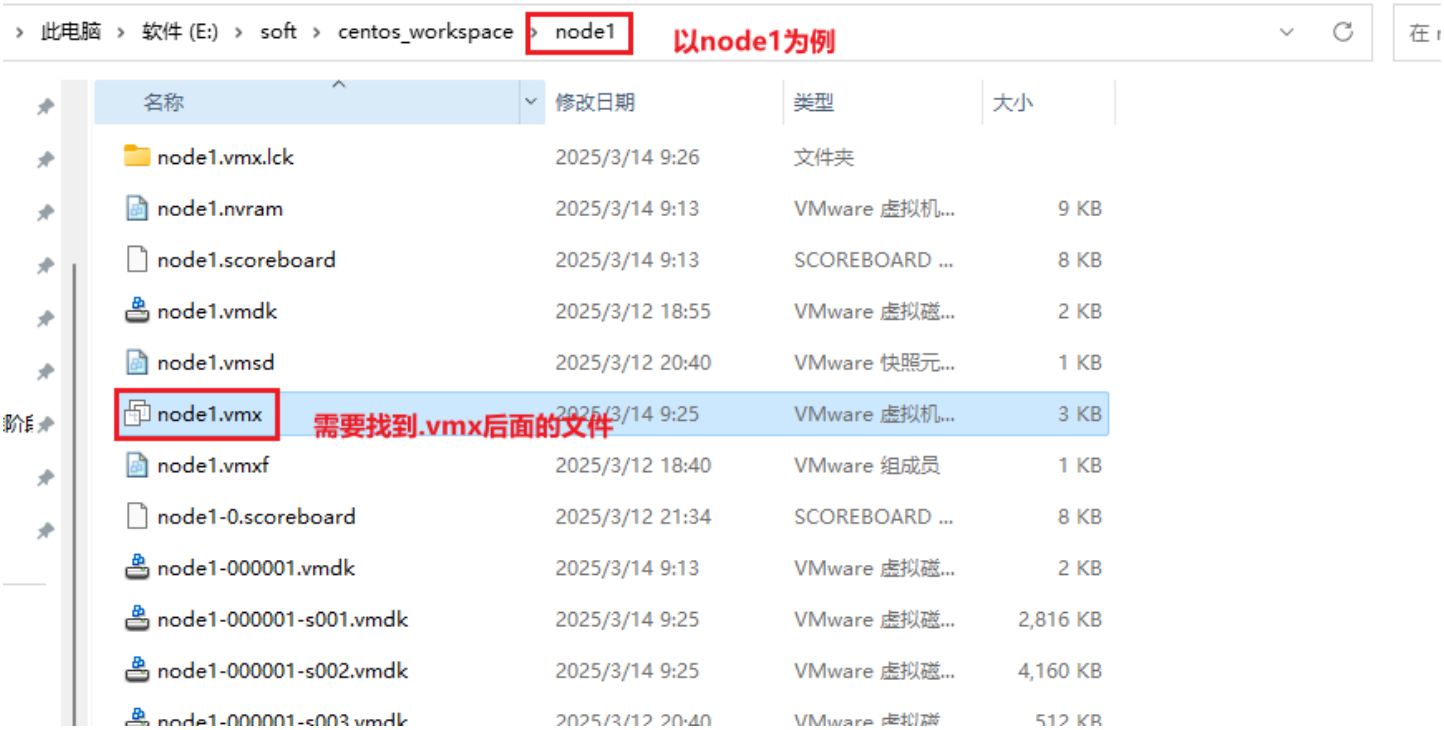
- 版本不兼容



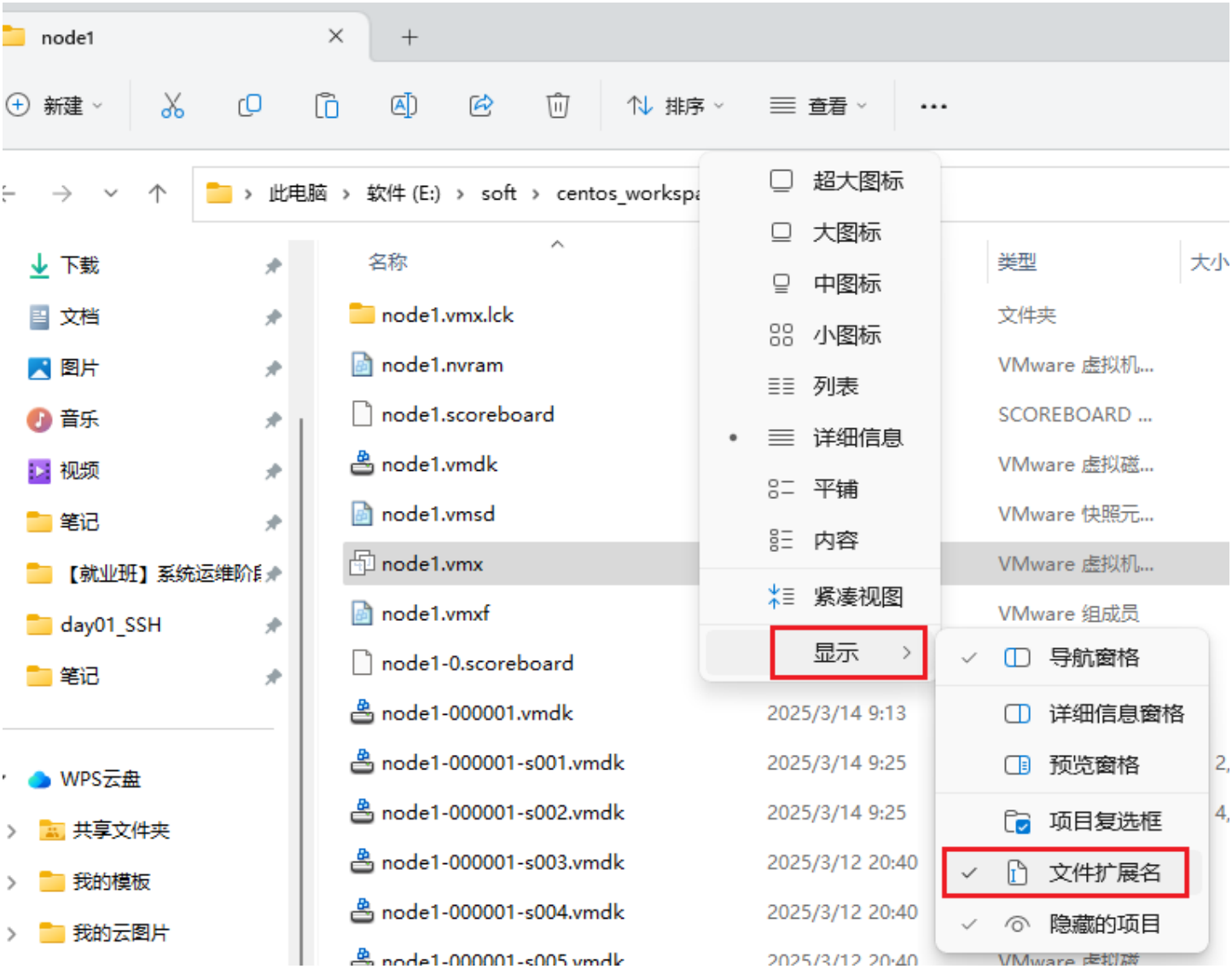
1 原因： 虚拟机在创建的时候，对应VMware版本， 和当前你需要挂载虚拟机对应VMware版本不一致

```
    导致， 一般都是由于虚拟机创建所对应VMware版本过高导致
2
3   VMware一般是可以向上兼容，但是无法进行向下兼容
```

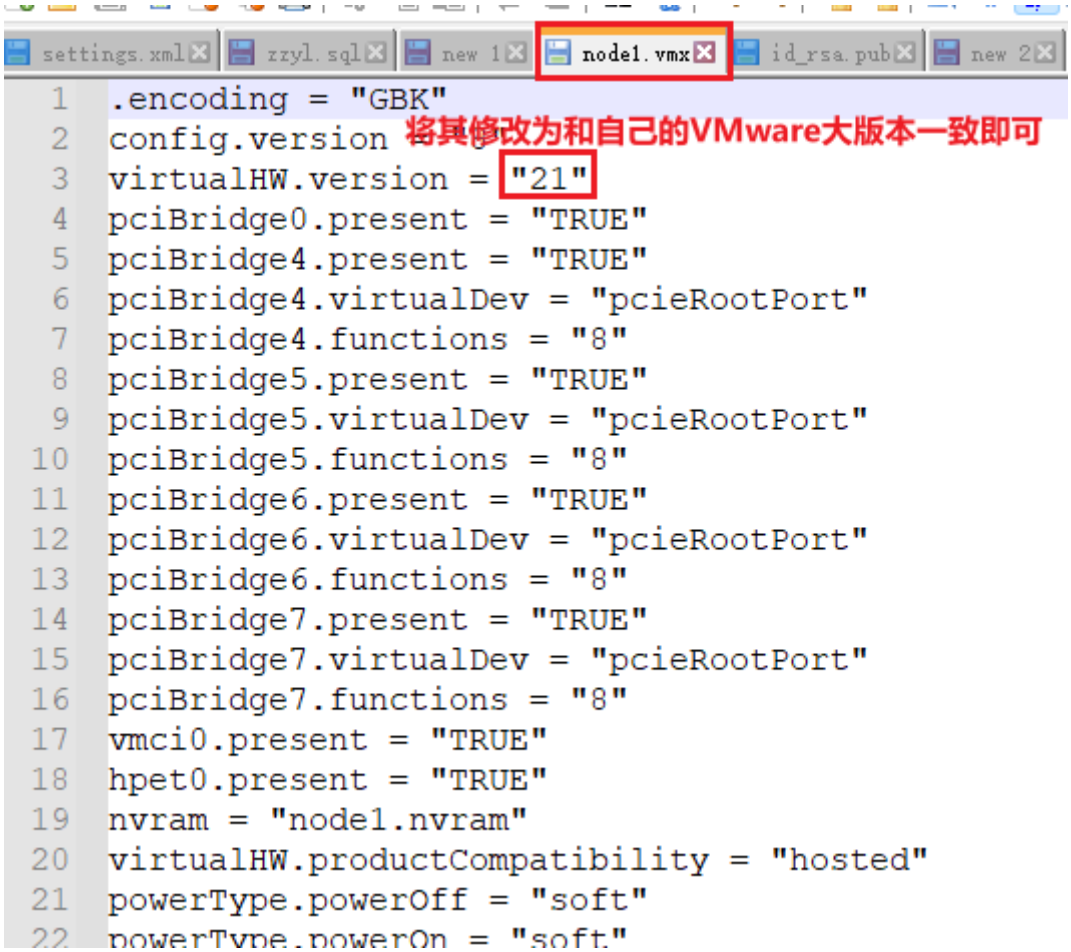
解决方案： 修改虚拟机的配置文件， 将其对应默认VMware版本号修改为与当前VMware版本一致



如果你的电脑不显示后缀：手动开启后缀显示

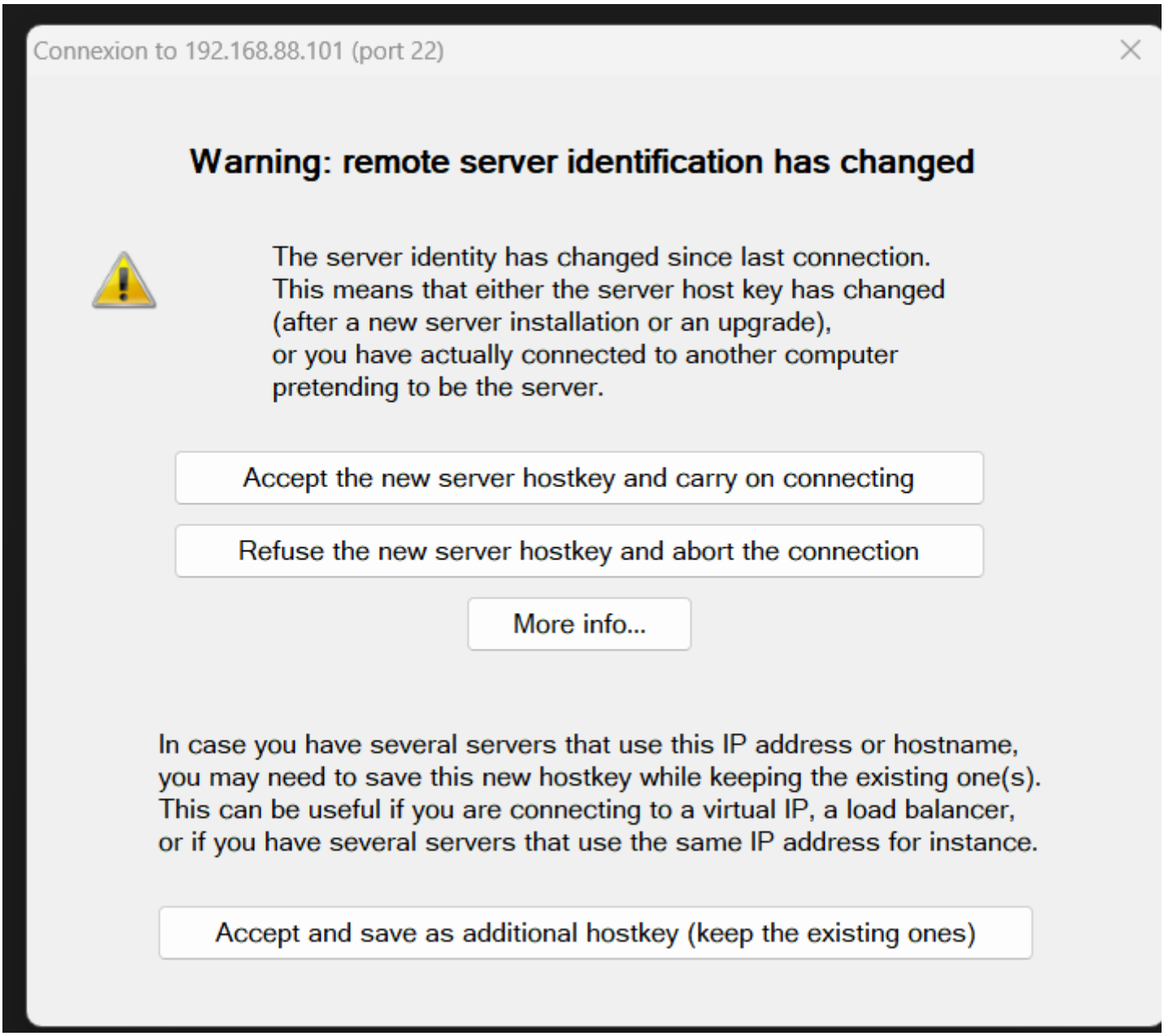


使用notepad++或其他的文件编辑器，但是不能是记事本，打开这个文件



更改完成后， 保存文件， 然后即可在VMware中开启虚拟机

说明2：



直接选择， 第一个选项， 接受即可

1. 克隆虚拟机

1.1 什么是克隆

克隆是创建一个虚拟机的完整副本，包括虚拟机的所有数据、配置和操作系统。克隆可以分为完整克隆和链接克隆两种：

- **完整克隆**：虚拟机的独立副本，克隆后与原始虚拟机无关联，适合需要长期使用的场景。
- 链接克隆：依赖于原始虚拟机，仅复制其差异部分，适合节省存储空间和快速创建测试环境，但需要保留原虚拟机。

> 克隆通常用于快速部署相同配置的多个虚拟机。

1.2 创建完整克隆

第一步：关闭当前操作系统



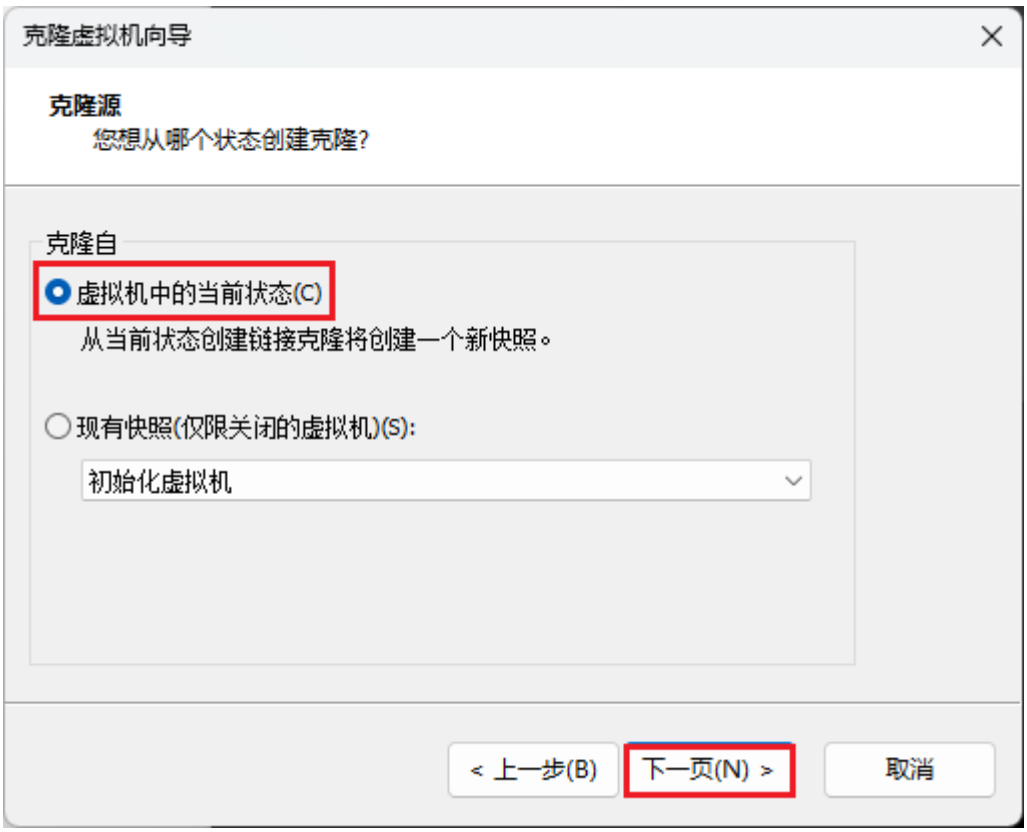
第二步：创建完整克隆



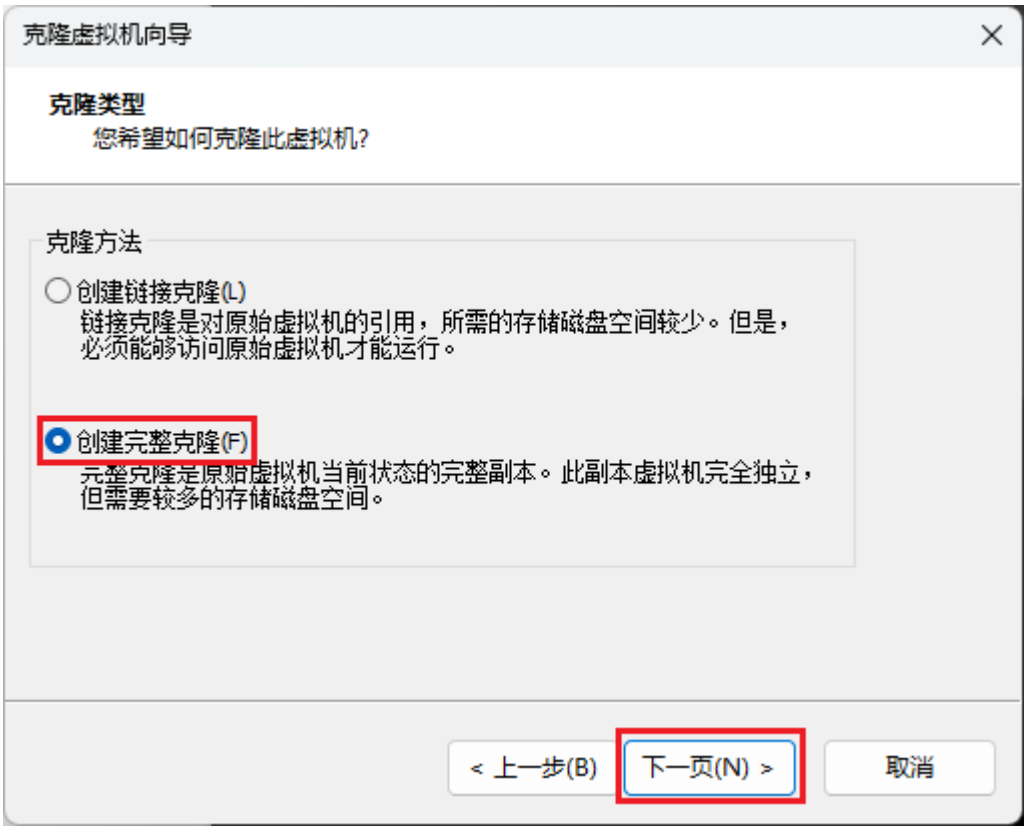
进入克隆设置向导



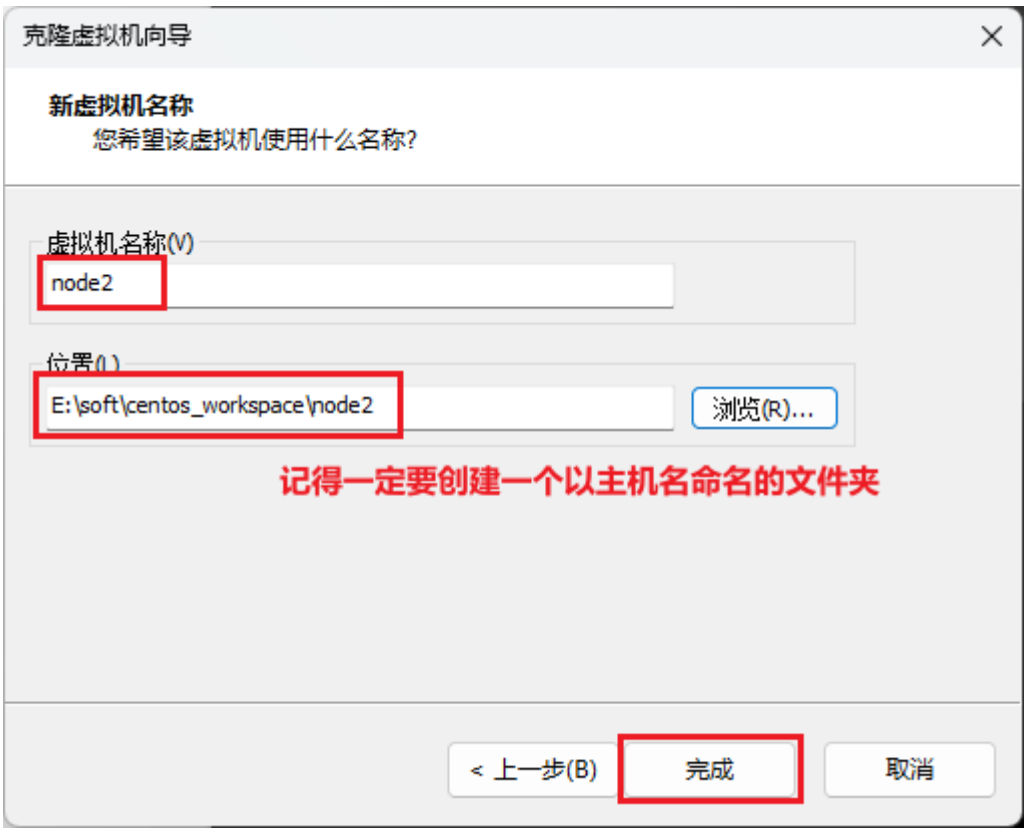
单击下一步，选择克隆源

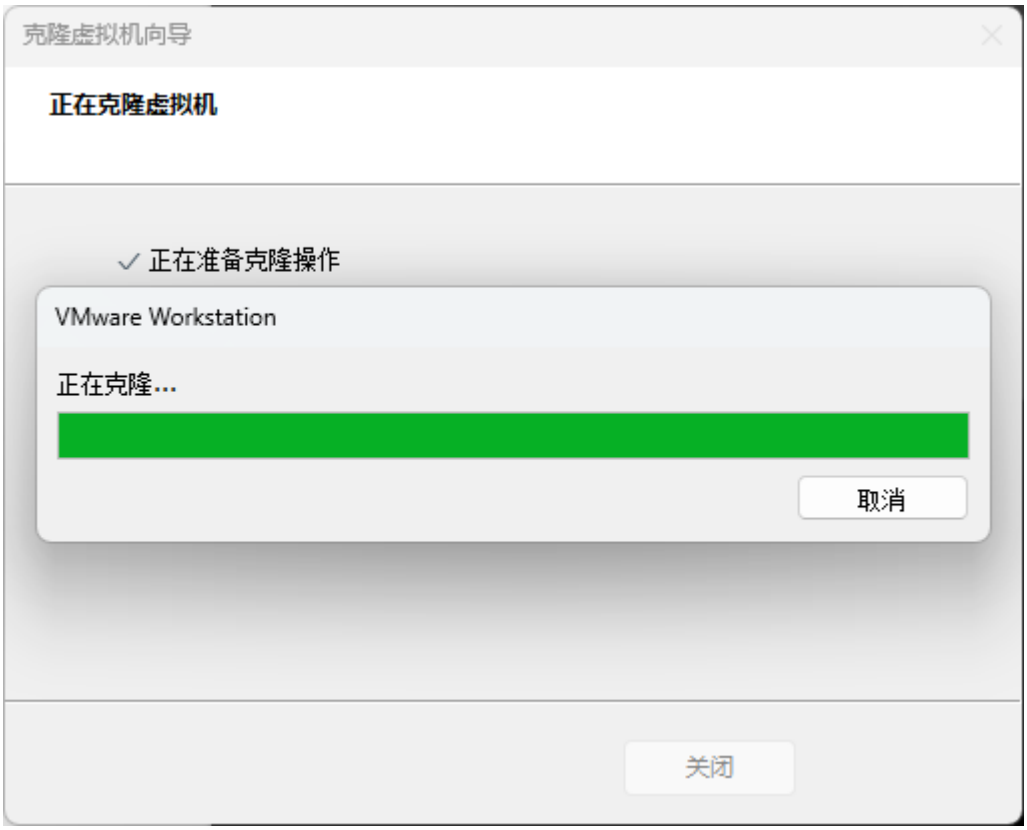


单击下一步，选择克隆类型 => 【完整克隆】



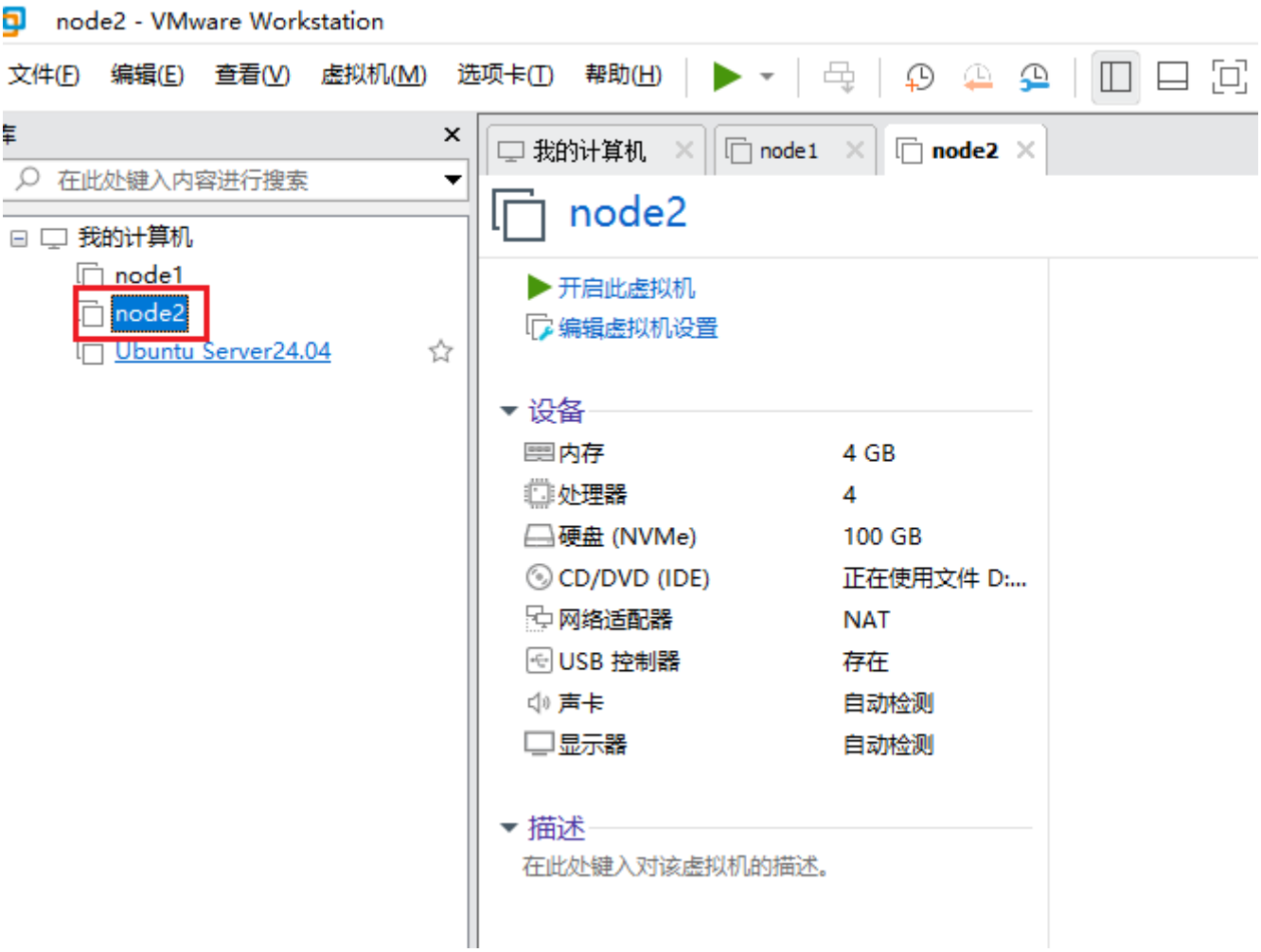
设置克隆后虚拟机的名称及存储位置




克隆虚拟机向导
正在克隆虚拟机
正在准备克隆操作
VMware Workstation
正在克隆...
取消
关闭

等待克隆完成：


克隆虚拟机向导
正在克隆虚拟机
正在准备克隆操作
正在创建完整克隆
完成
关闭

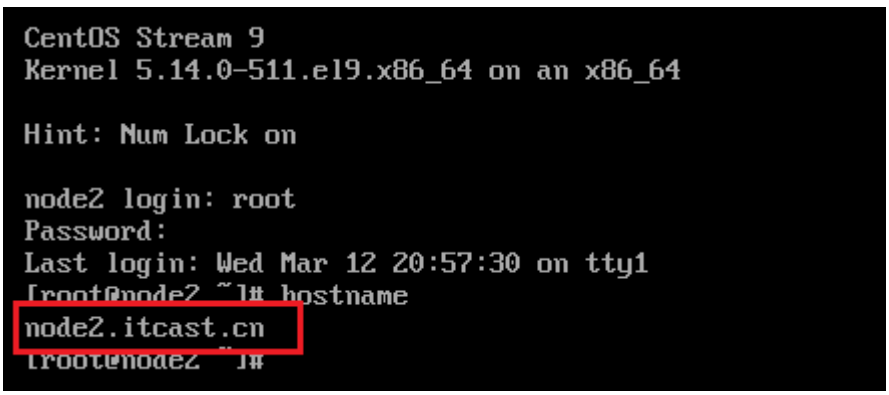


1.3 修改服务器配置

- 1- 修改主机名：node2

```
hostnamectl set-hostname node2.itcast.cn

关闭会话， 重新连接即可
```



- 2- 设置静态IP地址： 192.168.88.102/24

```
vi /etc/NetworkManager/system-connections/ens160.nmconnection
```

```
[connection]
id=ens160
uuid=32878fdb-b450-3d07-bece-bc533ac66c3e
type=ethernet
autoconnect-priority=-999
interface-name=ens160
timestamp=1741804943

[ethernet]

[ipv4]
method=manual
addresses=192.168.88.101/24
gateway=192.168.88.2
dns=8.8.8.8;114.114.114.114;

[ipv6]
addr-gen-mode=eui64
method=ignore

[proxy]
```

更改为：192.168.88.102/24

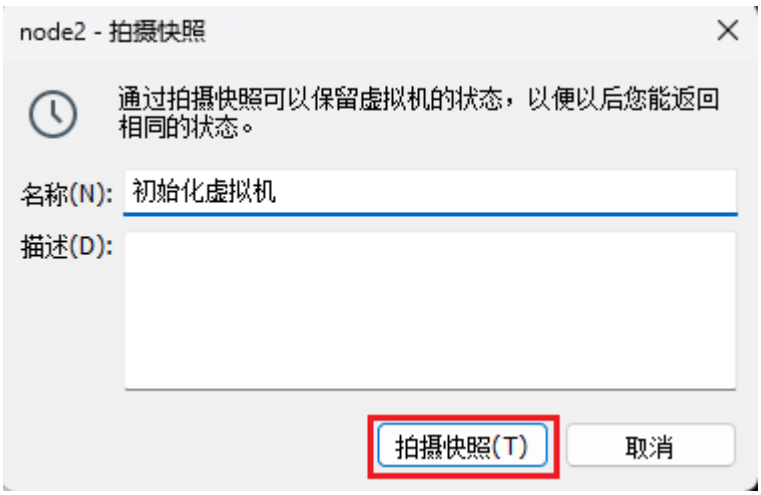
```
systemctl restart NetworkManager
nmcli connection down ens160 && nmcli connection up ens160

查看IP:
ip a
```

```
[root@node1 ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:7b:18:24 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 192.168.88.102/24 brd 192.168.88.255 scope global noprefixroute ens160
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe7b:1824/64 scope link
       valid_lft forever preferred_lft forever
[root@node1 ~]#
```

- 3- 对新服务器拍摄快照【先将服务器关机】





2. SSH基本概念

SSH（Secure Shell，安全外壳）是一种**网络安全协议**，旨在通过加密和认证机制实现安全的**远程访问**和**文件传输**等业务。

SSH支持多种身份验证方法，包括**密码认证**和**密钥认证**。密码认证是将用户名和密码发送给服务器进行认证，【重要】而密钥认证则使用**公钥**和**私钥**对进行身份验证，实现安全的免密登录。

```
公钥：
    公钥是可以公开分享的密钥，用于加密信息和验证数字签名。
    公钥需要被放置在远程服务器上用户账号的~/.ssh/authorized_keys文件中。

```

```
私钥：
    私钥是必须保密的密钥，用于解密信息和创建数字签名。
    私钥保存在客户端，通常位于用户主目录下的隐藏文件夹中，如~/.ssh/id_rsa。


注意：私钥非常重要，不能泄露或分享给其他人。一旦私钥被泄露，任何人都可以使用它来冒充你进行SSH连接。
```



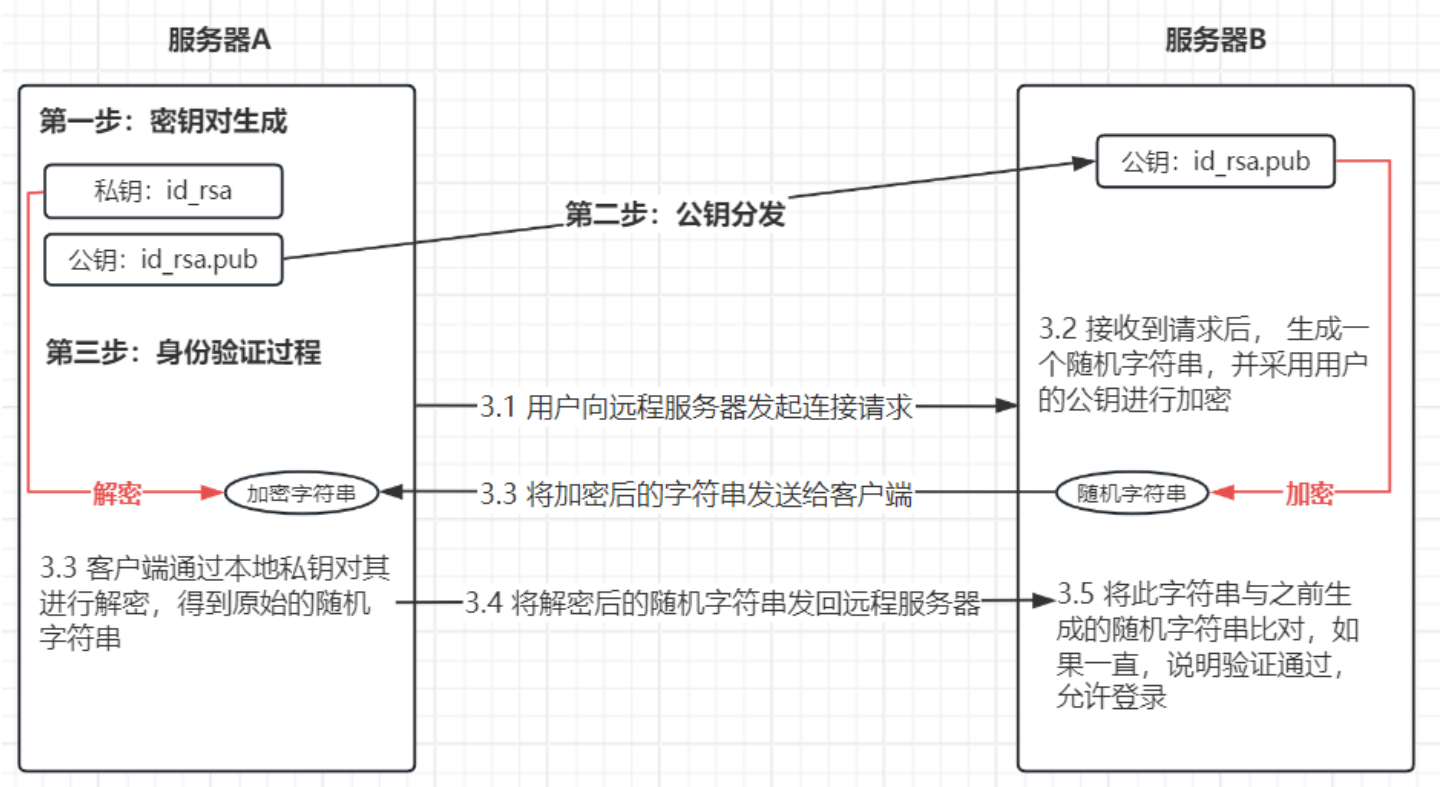
3. SSH工作原理

- 基本流程：

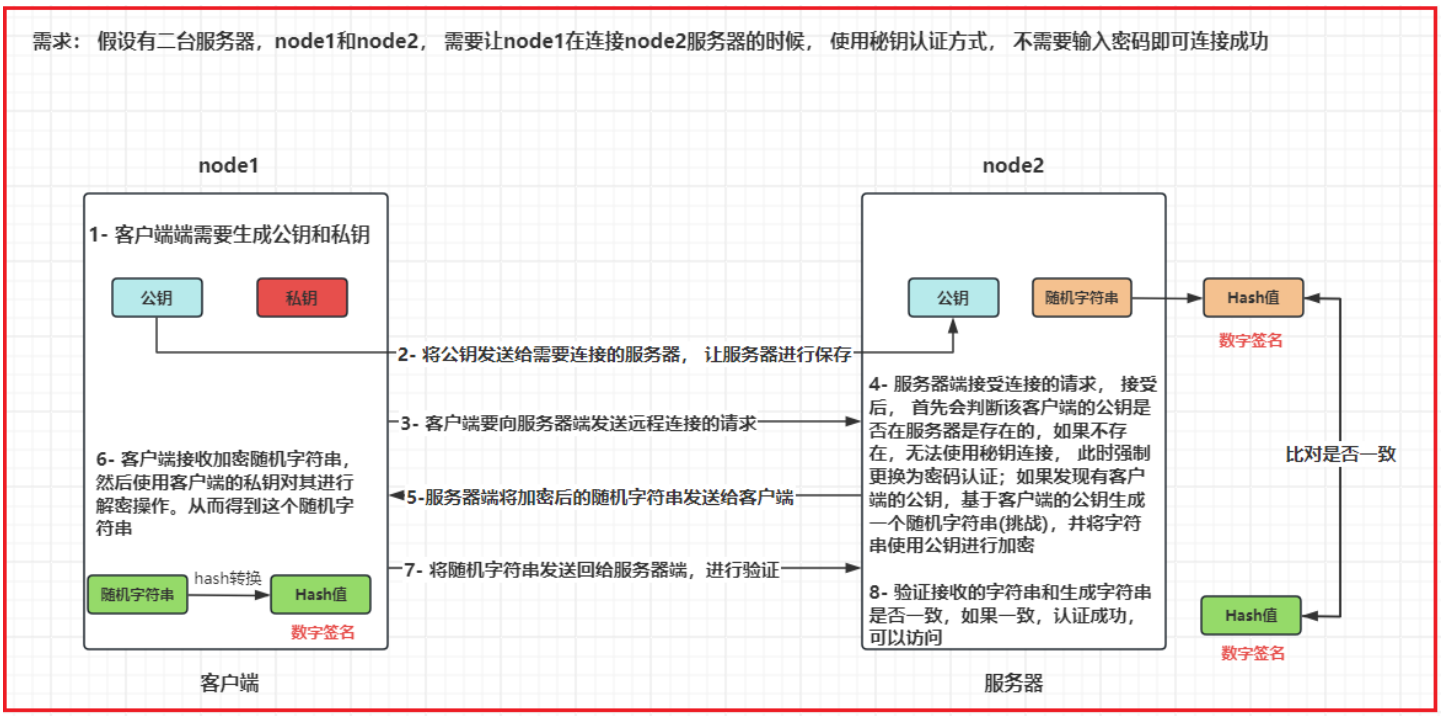
```
1- 密钥对生成：
    用户在本地机器上使用ssh-keygen命令生成一对密钥，即公钥和私钥。
    私钥必须保密，并存储在本地机器的~/.ssh/id_rsa文件中（或其他用户指定的位置）。
    公钥则可以公开，并存储在本地机器的~/.ssh/id_rsa.pub文件中。

2- 公钥分发：
    用户需要将生成的公钥文件（id_rsa.pub）的内容复制到远程服务器的~/.ssh/authorized_keys文件中。
    这通常通过使用ssh-copy-id命令或其他安全方式完成。

3- 身份验证过程：
    3.1 当用户尝试通过SSH登录到远程服务器时，SSH客户端会向远程服务器发起连接请求。
    3.2 远程服务器接收到连接请求后，会生成一个随机字符串（称为“挑战”），并使用用户的公钥对其进行加密，然后将加密后的字符串发送给客户端。
    3.3 客户端接收到加密后的字符串后，使用本地存储的私钥对其进行解密，得到原始的随机字符串。
    3.4 客户端将解密后的随机字符串发送回远程服务器。
    3.5 远程服务器接收到客户端发送的解密后的字符串后，会与之前生成的随机字符串进行比对。如果两者一致，说明客户端拥有与公钥对应的私钥，验证通过，允许用户登录。
```



课程中图：



4. SSH配置实战

4.1 安装SSH服务并配置

- 1- 检查SSH服务是否已安装

```
1   ssh -V : 检查SSH客户端版本，如果安装会显示版本信息
```

```
[root@node1 ~]# ssh -V
OpenSSH_8.7p1, OpenSSL 3.2.2 4 Jun 2024
[root@node1 ~]#
```

```
systemctl status sshd：  检查SSH服务状态，如果服务正在运行则说明已安装SSH服务器
```

```
[root@node1 ~]# systemctl status sshd
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: active (running) since Fri 2024-11-08 18:01:06 CST; 2h 30min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 852 (sshd)
      Tasks: 1 (limit: 22930)
     Memory: 11.5M
        CPU: 293ms
     CGroup: /system.slice/sshd.service
             └─852 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 08 20:04:58 node1.itcast.cn sshd[26787]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Nov 08 20:05:14 node1.itcast.cn sshd[26907]: Connection closed by 192.168.88.101 port 34848 [preauth]
Nov 08 20:05:21 node1.itcast.cn sshd[26940]: Connection closed by authenticating user root 192.168.88.101 port 40284 [preauth]
Nov 08 20:08:56 node1.itcast.cn sshd[27217]: Accepted publickey for root from 192.168.88.102 port 45682 ssh2: RSA SHA256:+dEgTMlFbI5f15fCVAa8c3iWA4spu4BaptKjriAWYz
Nov 08 20:08:56 node1.itcast.cn sshd[27217]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Nov 08 20:10:05 node1.itcast.cn sshd[27306]: Connection closed by authenticating user root 192.168.88.101 port 39266 [preauth]
Nov 08 20:10:05 node1.itcast.cn sshd[27319]: Connection closed by authenticating user root 192.168.88.101 port 39274 [preauth]
Nov 08 20:10:08 node1.itcast.cn sshd[27334]: Accepted password for root from 192.168.88.101 port 39280 ssh2
Nov 08 20:10:08 node1.itcast.cn sshd[27334]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Nov 08 20:10:08 node1.itcast.cn sshd[27334]: pam_unix(sshd:session): session closed for user root
[root@node1 ~]#
```

如果提示： command not found或类似的， 说明SSH服务未安装， 需要安装服务

```
[root@node1 ~]# systemctl status sshd
Unit sshd.service could not be found.
```

- 2- 安装SSH服务

```
dnf install -y openssh-server
dnf install -y openssh-clients

请注意，CentOS Stream 9使用的是dnf包管理器，而不是CentOS中的yum。因此，在安装软件包时，应使用dnf命令。
```

```
Upgrade  2 Packages

Total download size: 1.6 M
Is this ok [y/N]: y
Downloading Packages:
(1/3): openssh-server-8.7p1-44.el9.x86_64.rpm                                  3.1 MB/s | 460 kB     00:00
(2/3): openssh-8.7p1-44.el9.x86_64.rpm                                         1.3 MB/s | 460 kB     00:00
(3/3): openssh-clients-8.7p1-44.el9.x86_64.rpm                                 2.0 MB/s | 717 kB     00:00
--------------------------------------------------------------------------------------------------------------
Total                                                                          1.6 MB/s | 1.6 MB     00:00
CentOS Stream 9 - BaseOS                                                       1.6 MB/s | 1.6 kB     00:00
Importing GPG key 0x8483C65D:
 Userid     : "CentOS (CentOS Official Signing Key) <security@centos.org>"
 Fingerprint: 99DB 70FA E1D7 CE22 7FB6 4882 05B5 55B3 8483 C65D
 From       : /etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
Is this ok [y/N]: y
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                                    1/1
  Running scriptlet: openssh-8.7p1-44.el9.x86_64                                                        1/5
  Upgrading        : openssh-8.7p1-44.el9.x86_64                                                        1/5
  Running scriptlet: openssh-server-8.7p1-44.el9.x86_64                                                 2/5
  Installing       : openssh-server-8.7p1-44.el9.x86_64                                                 2/5
  Running scriptlet: openssh-server-8.7p1-44.el9.x86_64                                                 2/5
Created symlink /etc/systemd/system/multi-user.target.wants/sshd.service → /usr/lib/systemd/system/sshd.service.

  Upgrading        : openssh-clients-8.7p1-44.el9.x86_64                                                3/5
  Running scriptlet: openssh-clients-8.7p1-44.el9.x86_64                                                3/5
  Running scriptlet: openssh-clients-8.7p1-43.el9.x86_64                                                4/5
  Cleanup          : openssh-clients-8.7p1-43.el9.x86_64                                                4/5
  Cleanup          : openssh-8.7p1-43.el9.x86_64                                                        5/5
  Running scriptlet: openssh-8.7p1-43.el9.x86_64                                                        5/5
[  436.623037] systemd-rc-local-generator[1462]: /etc/rc.d/rc.local is not marked executable, skipping.
  Verifying        : openssh-server-8.7p1-44.el9.x86_64                                                 1/5
  Verifying        : openssh-8.7p1-44.el9.x86_64                                                        2/5
  Verifying        : openssh-8.7p1-43.el9.x86_64                                                        3/5
  Verifying        : openssh-clients-8.7p1-44.el9.x86_64                                                4/5
  Verifying        : openssh-clients-8.7p1-43.el9.x86_64                                                5/5

Upgraded:
  openssh-8.7p1-44.el9.x86_64                          openssh-clients-8.7p1-44.el9.x86_64
Installed:
  openssh-server-8.7p1-44.el9.x86_64

Complete!
```

遇到 y/n 都输入y

- 3- 启动SSH相关服务

```
systemctl start sshd   -- 启动服务
systemctl enable sshd  -- 将服务添加到自启动项，下次开机就自动启动了
```

```
[root@node1 ~]# systemctl start sshd
[root@node1 ~]# systemctl enable sshd
[  636.961935] systemd-rc-local-generator[1784]: /etc/rc.d/rc.local is not marked executable, skipping.
```

关于SSH配置文档相关参数：

SSH配置文件所在路径：/etc/ssh/sshd_config

```
vi /etc/ssh/sshd_config

核心配置说明：
Port：指定SSH服务监听的端口号，默认为22，可以修改为其他端口以增强安全性
PermitRootLogin：控制是否允许root用户通过SSH登录，建议设置为no以提高安全性
PasswordAuthentication：控制是否允许使用密码进行认证，如果启用了密钥认证，可以将其设置为no
ChallengeResponseAuthentication 和UsePAM：通常与密码认证相关，如果禁用了密码认证，可以将其设置为no
AllowUsers 和 DenyUsers：分别指定允许和拒绝通过SSH登录的用户列表

```

```
如果修改了相关配置， 需要重新加载SSH服务配置，使其生效：

命令: systemctl reload sshd
```

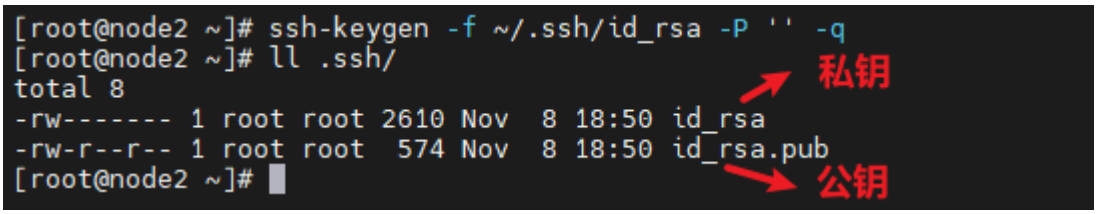
4.2 如何配置SSH免密登录

场景说明： 目前有node1和node2二台服务器，需要实现node2访问node1时，可以免密访问

- 1- 在node2中生成秘钥对

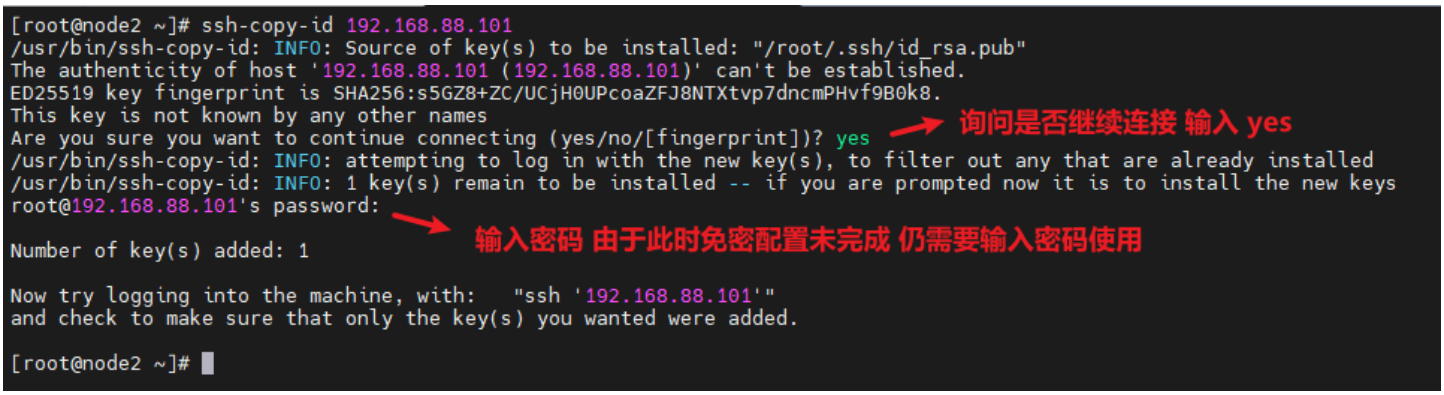
```
ssh-keygen -f ~/.ssh/id_rsa -P '' -q

说明:
-f ： 指定私钥文件的保存位置 （默认为 ~/.ssh/id_rsa ）
-P ： 指定设置私钥的密码 （一般为空，不设置）
-q ： 表示静默执行，此时大部分的信息不回显示，仅必要信息才会输出
```



- 2- 发送公钥到node1节点

```
ssh-copy-id 192.168.88.101
```

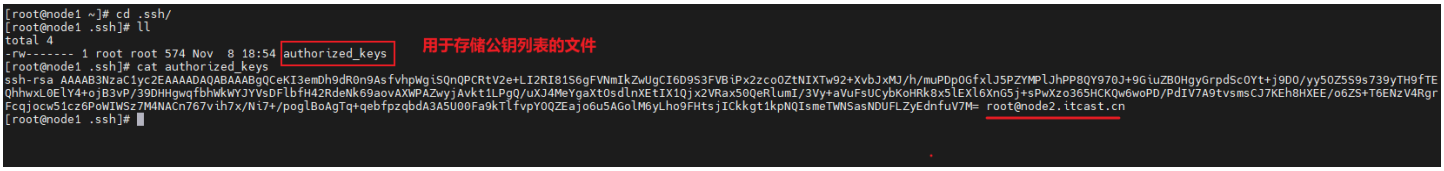


如果不希望提示是否继续连接

```
ssh-copy-id -o StrictHostKeyChecking=no 192.168.88.101
```

在node1节点，查看是否发送成功

默认位置： ~/.ssh/authorized_keys



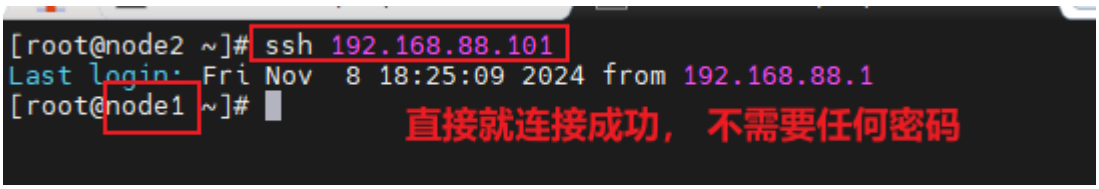
- 3- 在node2 通过 ssh命令连接node1，尝试是否可以免密码访问成功

```
ssh 192.168.88.101

或者
ssh root@192.168.88.101

说明:
root@192.168.88.101
用户名@连接主机

如果省略了用户名，默认使用当前服务器所登录的用户来连接目标服务器
```

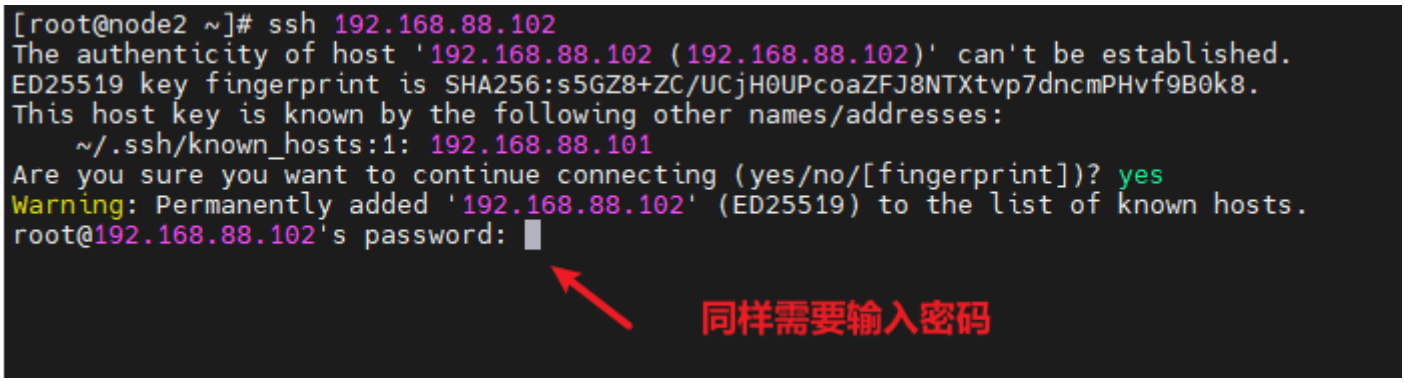


思考： 如果node1访问node2是否可以呢?



作业： 尝试完成 node1免密访问node2操作

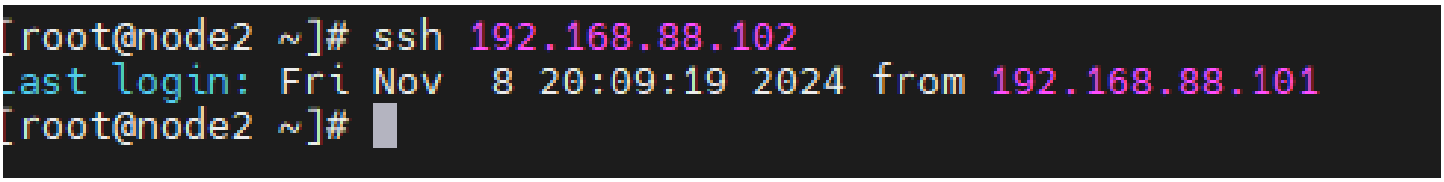
注意事项： 尝试通过SSH 自己访问自己，观察会有什么现象




```
原因：  因为并没有把公钥信息添加到自己节点中专门用于放置公钥的授权文件中

操作：在node2执行，  将公钥发送到自己的授权文件中
ssh-copy-id 192.168.88.102

操作：在node1执行，  将公钥发送到自己的授权文件中
ssh-copy-id 192.168.88.101
```



5. 基于SSH实现发送命令到远程服务器

SSH远程执行命令是通过SSH协议连接到远程服务器，并在远程服务器上执行指定的命令或脚本，而无需直接登录到服务器的终端。

有什么价值呢?

1. **高效管理**：无需登录每台服务器，即可快速执行命令，特别适合管理多台服务器。
2. **自动化运维**：结合脚本或工具，实现批量操作、定时任务和自动化部署。
3. **快速故障排查**：实时查看日志、监控状态，快速定位和解决问题。
4. **安全性**：通过加密通信执行命令，避免敏感信息泄露。
5. **节省时间**：无需物理接触服务器，即可完成维护任务，特别适合分布式或云环境。

- 基本语法

```
ssh [选项] username@remote_host "command [&]"

说明:
username：远程服务器的用户名。
remote_host：远程服务器的IP地址或域名。
```

```
command：要在远程服务器上执行的命令

& ： 异步执行， 在远程服务器后台运行， SSH不会等待命令完成，会立即返回并关闭连接

选项：
-t： 如果远程命令需要交互式终端（例如 sudo 命令），可以使用 -t 选项
-p： 如果远程服务器的SSH服务运行在非默认端口（默认是22），可以使用 -p 选项指定端口
-o： 使用 -o 选项来指定SSH配置选项，例如禁用主机密钥检查（StrictHostKeyChecking=no）
-A： 需要转发认证代理连接（例如使用SSH密钥进行跳板机连接），可以使用 -A 选项
```

相关操作

```
需求一： 在node1中远程查看node2的主机名
ssh root@192.168.88.102 "hostname"

需求二： 在node1中，远程执行进入root用户家目录，并在此目录下创建 a.txt文件
说明：如果是执行多个命令，可以使用分号 或 && 来分隔命令
    分号： 仅分隔命令， 不管前面得命令是否执行成功， 后面的命令依然会执行
    && ： 将多个命令合并为一个命令， 前面得命令如果执行失败， 后面的命令就无法执行

ssh root@192.168.88.102 "cd ~ && touch a.txt"
```

6. 基于SSH进行免密数据传输（SCP）

SCP（Secure Copy Protocol）是基于 SSH 协议的安全文件传输工具，可用于在本地和远程服务器之间传输文件或目录。通过 SCP，数据在传输过程中会被加密，确保安全性。

SCP的基本语法结构：

```
scp [选项] [源文件路径] [目标路径]

源文件路径：本地文件或远程服务器上的文件。
目标路径：本地路径或远程服务器上的路径。

常用选项：
-r          递归复制整个目录（传输目录时必用）
-P          指定 SSH 连接的端口号（默认是 22）
-i          指定 SSH 使用的私钥文件（用于免密登录）
-q          静默模式，不显示进度条和调试信息
-C          开启压缩传输，提高传输速度（适合大文件或低速网络）
-l          限制传输速度（单位为 Kbit/s，例如 -l 1000 表示限制为 1Mbps）
```

SCP的基本用法：

```
1. 从本地传输到远程
    scp /local/path/file.txt user@remote:/remote/path/
2. 从远程传输到本地
    scp user@remote:/remote/path/file.txt /local/path/

3. 递归传输整个目录
    scp -r /local/path/dir user@remote:/remote/path/
```

相关案例：

```
1- 在node1 /root目录下，构建 ssh_test目录，并在此目录下创建a.txt b.txt文件，向文件中随机添加一些数据
cd ~
mkdir -p ./ssh_test
cd /root/ssh_test
touch a.txt b.txt
echo "a.txt 11111111" > a.txt
echo "b.txt 22222222" > b.txt


2- 通过SCP命令， 在node1完成将该目录拷贝到 node2的 / 目录下
scp -r /root/ssh_test/ root@192.168.88.102:/

3- 通过SCP命令， 在node2的家目录完成将远端/root/ssh_test/a.txt 拷贝到当前目录操作
cd ~
scp root@192.168.88.101:/root/ssh_test/a.txt ./
```

7. 综合案例

要求， 在node1节点中安装JDK， 完成安装后， 基于SCP快速完成node2节点的JDK安装操作

- 第一步： 完成在node1安装JDK

```
以下操作均在node1节点执行：
# 1- 创建目录 并切换至目录下
mkdir -p /export/software
cd /export/software

# 2- 上传JDK的压缩包
使用 mx工具， 将提供jdk的安装包上传到/export/software
```

```
# 4- 解压JDK压缩包
cd /export/software
tar -xzf jdk-8u241-linux-x64.tar.gz -C /opt

# 5- 验证是否ok
cd /opt/jdk1.8.0_241/bin

./java -version
./javac

测试是否正常， 正常是可以看到jdk1.8版本的字样， 同时javac执行也可以看到一些相关的参数提示

#6- 配置环境变量
vim /etc/profile

输入G，跳转文件末尾， 然后输入o 在下一行输入内容

#将以下内容，包含注释， 全部拷贝到文件中
# JAVA_HOME
export JAVA_HOME=/opt/jdk1.8.0_241
export PATH=$JAVA_HOME/bin:$PATH
export CLASSPATH=$JAVA_HOME/lib:$CLASSPATH

按下ecs键， 输入 :x 保存退出

最后，执行重新加载配置文件
source /etc/profile


最终校验： 在任意的路径下执行
java -version
javac

测试是否正常， 正常是可以看到jdk1.8版本的字样， 同时javac执行也可以看到一些相关的参数提示


额外注意：关于在修改环境变量的时候， 请注意， java的路径地址是否正确， 需要校验
```

- 第二步：通过scp命令 快速完成node2的JDK安装操作

```
在node1执行：
```

```
cd /opt
scp -r jdk1.8.0_241/ root@192.168.88.102:$PWD

说明:  $PWD 常量值  表示当前所在的路径

所以上述的命令等同于:   scp -r jdk1.8.0_241/ root@192.168.88.102:/opt

SCP /etc/profile root@192.168.88.102:/etc/

在node2执行:
source /etc/profile

测试:在任意的路径下执行
java -version
javac

测试是否正常,  正常是可以看到jdk1.8版本的字样,  同时javac执行也可以看到一些相关的参数提示
```

8. 【实战】基于Ubuntu完成SSH服务配置

需求:node1(centos stream)、node2(centos stream)、Ubuntu Server01(Ubuntu)之间的SSH免密配置操作

要求: 分别为node1和node2构建itheima用户,设置密码为123456, 然后三台服务器全部基于此账号完成SSH配置操作

```
第一步:  需要让三个服务器全部都使用itheima用户登录服务器
1.1 开启三台服务器(node1 node2  ubuntu server)
1.2 node1和node2在默认情况下,只有root用户,  没有其他用户,  此时需要先创建用户
   1.2.1:node1和node2先使用root用户登录执行以下命令
       useradd itheima
       passwd itheima
       设置密码为:123456
       设置完成后,  使用su命令,测试使用可以正常切换至itheima
   1.2.2 在MX工具上,  将原有二个会话关闭,  然后创建新的使用itheima账号的会话窗口,同时使用itheima连接Ubuntu server
```

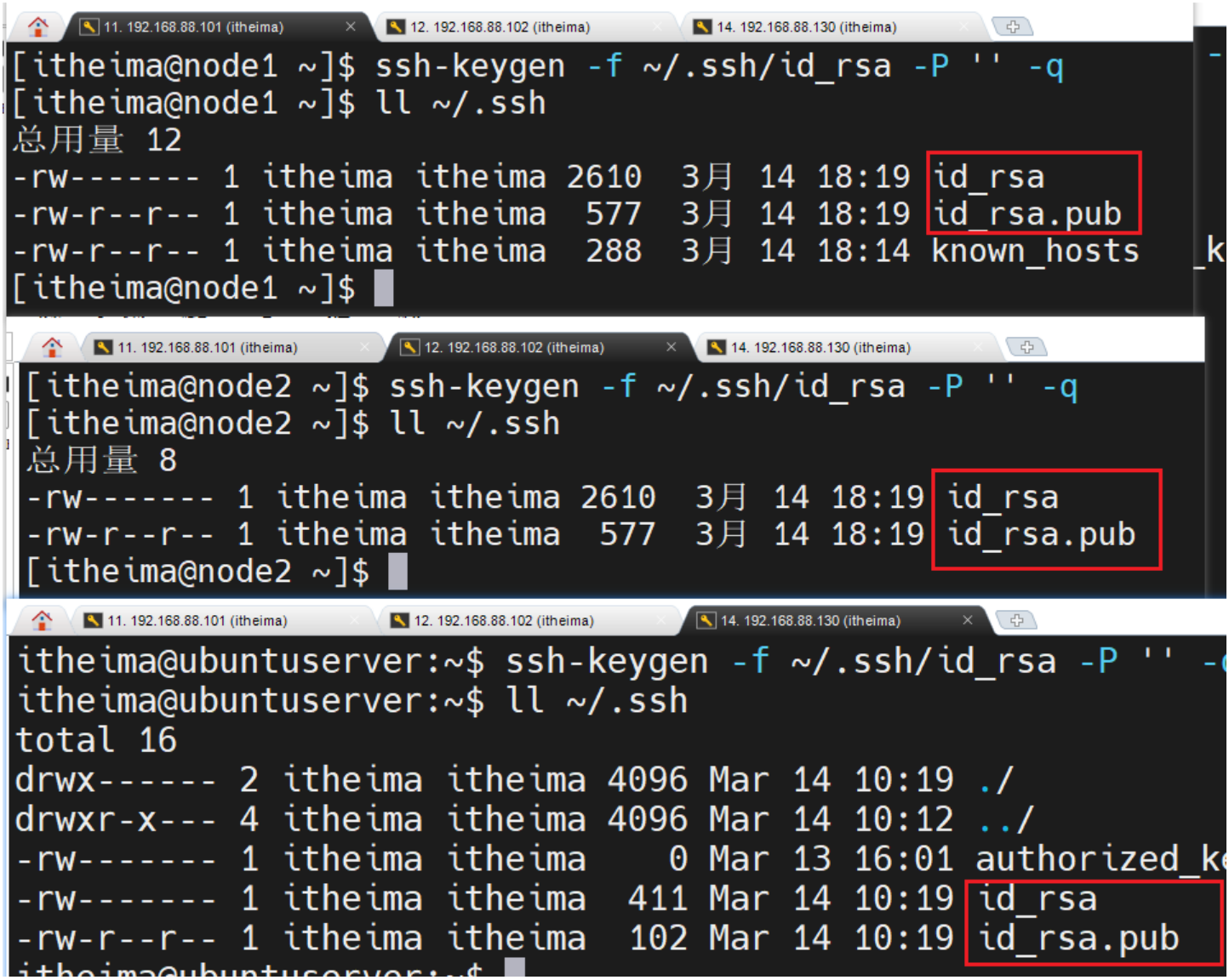



```
第二步：需要让三台服务器都要配置免密操作
    分析：
        node1 连接 node1 node2 Ubuntu 不需要输入密码
        node2 连接 node1 node2 Ubuntu 不需要输入密码
        Ubuntu 连接 node1 node2 Ubuntu 不需要输入密码
    发现， 每一个节点都要连接三个服务器节点， 意味每个节点都应该有三个节点公钥

    结论： 只需要让每一个节点都拥有三个节点公钥信息即可

2.1 三个节点统一生成在itheima账号下， 生成公钥和私钥
# 三个节点都要执行
ssh-keygen -f ~/.ssh/id_rsa -P '' -q

2.2 校验每个节点的itheima用户下的.ssh目录中是否已经有了公钥和私钥
# 三个节点
ll ~/.ssh
```




```
2.3 将自己的公钥发送给三个节点:
# 以下命令三个节点都要执行【一行一行执行】
ssh-copy-id 192.168.88.101
ssh-copy-id 192.168.88.102
ssh-copy-id 192.168.88.130

2.4 校验:  查看每个节点公钥授权文件,  观察是否有三个节点的公钥
# 以下命令三个节点都要执行
cat ~/.ssh/authorized_keys
```

node1节点

```
[itheima@node1 ~]$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQC1LX8saCK/WI919tGylhFk7jUiuGTR8oIxyTH1xTg42MPm/sSbgEvh2JKRiFIqR3GagYO
ed5fUv+HS89o96mhquuAU/xwLX0Pb+stpANPyBLlizTJjj76v+ML3sM/ESUSEgEoFn2XzogVubvYez+cCK0Sr30HKKQGK7Jqc3YIHk5B2UV
RwfBpsRygJyo6cZf9KB3XR/qcEH9GBHL7Iy3kh13uuhl3ooSLHUzfntth9hc78BGd/iRt2u7b7Nuyrxgi655iQxTWVSaNC+Wk8RwpOuy2Oz
Xrka1RMrNjUTwv9C/5XavW9C6wqK6oIDlbWHj+O1cdunZVjUdRoOR9Axc3fKzFfLWdGJQEmd+CbO0aX0JxknGNe6tOaQFXV/DMm9YvQ7FDl
O70ruGGlfBkFd/sBLXmE09bls0aQqg4MVnl/ikIdze9WOYz4XuG9ht9osjAEfi7IIU7ntxi9XixuUEIPKjBFcFy7WgkymsnoM2bWy7H5mt1
C97qcOM0JPFmi+zc= itheima@node1.itcast.cn
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDNQTcOYl/1nidYWAcfnc3tgGS+SnQQSxG5XKvRG4jvbQX1mUhsmUNXJfTRkH0KZyjEtsD
lruslIu8AsMhM+NCvTmVFSfHyqaoVs2W1mT5NVj0bs1hAE0mHUfXSAbLd56pcwKYowMV2ov1hugPWxs6HZmMF8bT8eQ9+EUjEmyWTMO54OI
meCVPTEehAzXCrNPSkikMy5RKV5IAFPgQhhcm/jJwDIi0HnTCg9BhwCFT1eFb5SVlromjEGdYFDgVP9kbTi0uWFfyuf/7+XqBQ+Fk7cSmdF
EhA8jcxrYavp4efennWCRH+fatgm/b3J3PdUcYwSpLOQp3ZHgCywR8sJN+GNzKGyJvNDNdIIYS0JdEs+Ik4Oe+Me2wqIETA9+wSjPcoWt/+
1Tw/8quQJGXpUdU4JWrE2XtuNBR/V0oloOgJgJVSartd1ONuuK00MHO7f5ZraDvzOAaQYgWVn00jCFScJdwyJrP465wx8KbNLPq1XSm/PeG
zRwGXaPKBQf2FzMk= itheima@node2.itcast.cn
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJxd6u0Vx9sbyIZ6vDlR23/jEmGhANkSJKd0b5vBDOuv itheima@ubuntuserver
[itheima@node1 ~]$
```

node2节点

```
[itheima@node2 ~]$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQC1LX8saCK/WI919tGylhFk7jUiuGTR8oIxyTH1xTg42MPm/sSbgEvh2JKRiFIqR3GagYO
ed5fUv+HS89o96mhquuAU/xwLX0Pb+stpANPyBLlizTJjj76v+ML3sM/ESUSEgEoFn2XzogVubvYez+cCK0Sr30HKKQGK7Jqc3YIHk5B2UV
RwfBpsRygJyo6cZf9KB3XR/qcEH9GBHL7Iy3kh13uuhl3ooSLHUzfntth9hc78BGd/iRt2u7b7Nuyrxgi655iQxTWVSaNC+Wk8RwpOuy2Oz
Xrka1RMrNjUTwv9C/5XavW9C6wqK6oIDlbWHj+O1cdunZVjUdRoOR9Axc3fKzFfLWdGJQEmd+CbO0aX0JxknGNe6tOaQFXV/DMm9YvQ7FDl
O70ruGGlfBkFd/sBLXmE09bls0aQqg4MVnl/ikIdze9WOYz4XuG9ht9osjAEfi7IIU7ntxi9XixuUEIPKjBFcFy7WgkymsnoM2bWy7H5mt1
C97qcOM0JPFmi+zc= itheima@node1.itcast.cn
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDNQTcOYl/1nidYWAcfnc3tgGS+SnQQSxG5XKvRG4jvbQX1mUhsmUNXJfTRkH0KZyjEtsD
lruslIu8AsMhM+NCvTmVFSfHyqaoVs2W1mT5NVj0bs1hAE0mHUfXSAbLd56pcwKYowMV2ov1hugPWxs6HZmMF8bT8eQ9+EUjEmyWTMO54OI
meCVPTEehAzXCrNPSkikMy5RKV5IAFPgQhhcm/jJwDIi0HnTCg9BhwCFT1eFb5SVlromjEGdYFDgVP9kbTi0uWFfyuf/7+XqBQ+Fk7cSmdF
EhA8jcxrYavp4efennWCRH+fatgm/b3J3PdUcYwSpLOQp3ZHgCywR8sJN+GNzKGyJvNDNdIIYS0JdEs+Ik4Oe+Me2wqIETA9+wSjPcoWt/+
1Tw/8quQJGXpUdU4JWrE2XtuNBR/V0oloOgJgJVSartd1ONuuK00MHO7f5ZraDvzOAaQYgWVn00jCFScJdwyJrP465wx8KbNLPq1XSm/PeG
zRwGXaPKBQf2FzMk= itheima@node2.itcast.cn
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJxd6u0Vx9sbyIZ6vDlR23/jEmGhANkSJKd0b5vBDOuv itheima@ubuntuserver
[itheima@node2 ~]$
```

node3节点

```
itheima@ubuntuserver:~$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQC1LX8saCK/WI919tGylhFk7jUiuGTR8oIxyTH1xTg42MPm/sSbgEvh2JKRiFIq
R3GagYOed5fUv+HS89o96mhquuAU/xwLX0Pb+stpANPyBLlizTJjj76v+ML3sM/ESUSEgEoFn2XzogVubvYez+cCK0Sr30HKKQGK
7Jqc3YIHk5B2UVRwfBpsRygJyo6cZf9KB3XR/qcEH9GBHL7Iy3kh13uuhl3ooSLHUzfntth9hc78BGd/iRt2u7b7Nuyrxgi655iQ
xTWVSaNC+Wk8RwpOuy2OzXrka1RMrNjUTwv9C/5XavW9C6wqK6oIDlbWHj+O1cdunZVjUdRoOR9Axc3fKzFfLWdGJQEmd+CbO0aX
0JxknGNe6tOaQFXV/DMm9YvQ7FDlO70ruGGlfBkFd/sBLXmE09bls0aQqg4MVnl/ikIdze9WOYz4XuG9ht9osjAEfi7IIU7ntxi9
XixuUEIPKjBFcFy7WgkymsnoM2bWy7H5mt1C97qcOM0JPFmi+zc= itheima@node1.itcast.cn
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDNQTcOYl/1nidYWAcfnc3tgGS+SnQQSxG5XKvRG4jvbQX1mUhsmUNXJfTRkH0K
ZyjEtsDlruslIu8AsMhM+NCvTmVFSfHyqaoVs2W1mT5NVj0bs1hAE0mHUfXSAbLd56pcwKYowMV2ov1hugPWxs6HZmMF8bT8eQ9+
EUjEmyWTMO54OImeCVPTEehAzXCrNPSkikMy5RKV5IAFPgQhhcm/jJwDIi0HnTCg9BhwCFT1eFb5SVlromjEGdYFDgVP9kbTi0uW
Ffyuf/7+XqBQ+Fk7cSmdFEhA8jcxrYavp4efennWCRH+fatgm/b3J3PdUcYwSpLOQp3ZHgCywR8sJN+GNzKGyJvNDNdIIYS0JdEs
+Ik4Oe+Me2wqIETA9+wSjPcoWt/+1Tw/8quQJGXpUdU4JWrE2XtuNBR/V0oloOgJgJVSartd1ONuuK00MHO7f5ZraDvzOAaQYgWV
n00jCFScJdwyJrP465wx8KbNLPq1XSm/PeGzRwGXaPKBQf2FzMk= itheima@node2.itcast.cn
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJxd6u0Vx9sbyIZ6vDlR23/jEmGhANkSJKd0b5vBDOuv itheima@ubuntuserve
r
itheima@ubuntuserver:~$
```

```
最后， 依次通过： ssh 主机地址 连接各个服务器， 观察是否需要输入密码

建议方案， 一个节点 一个节点依次测试， 确保不漏测

以node1为例：
ssh 192.168.88.101
通过exit退回 --> node1
ssh 192.168.88.102
通过exit退回 --> node1
ssh 192.168.88.130
```

```
11   通过exit退回 --> node1
```

如何恢复原状呢?

方式一： 全部使用快照， 直接恢复为初始化状态（centos stream） 【解决根本问题】

方式二： 删除所有用户下的 .ssh目录

rm -rf ~/.ssh 【Ubuntu server适用，其他服务器也可以使用， 但是建议直接快照方式】